

WEEK 08

Date: 11/09/2026

Title: Employee Salary Management using Interface and Runtime Polymorphism

Objective / Aim

To design and implement an **Employee Salary Management System** in Java using **interfaces, classes, objects, String operations, and runtime polymorphism** to display salary details of regular employees, contract employees, and vendors.

System Requirements

Hardware

- Standard PC/Laptop
- Minimum 4 GB RAM
- Minimum 500 MB free storage

Software

- Operating System: Windows / Linux / macOS
 - Java Development Kit (JDK 17 or later)
 - Text Editor or IDE such as VS Code, Eclipse, IntelliJ IDEA, or NetBeans
 - Command Prompt / Terminal
-

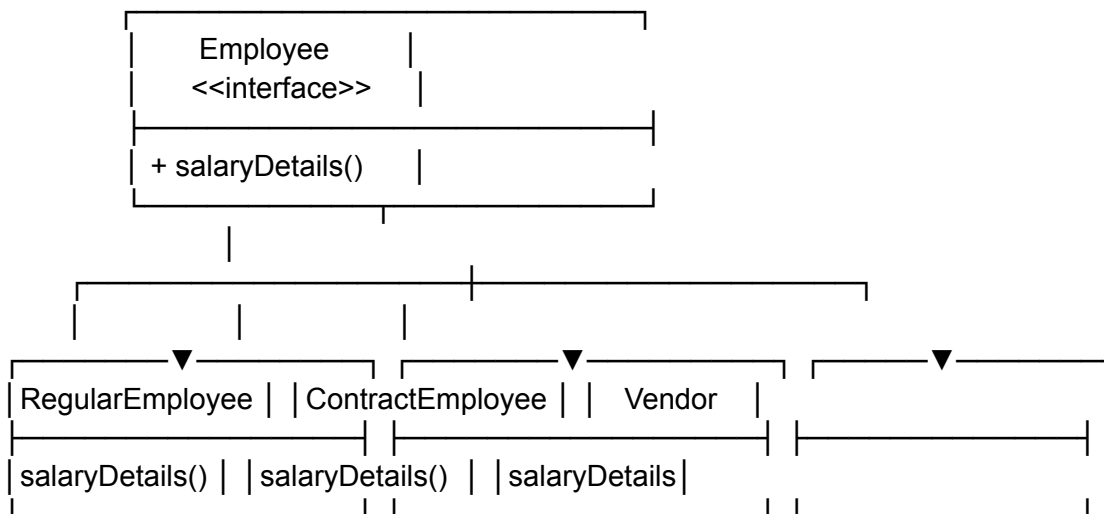
Algorithm / Logic Description

1. Create an interface named `Employee`.
2. Define the `salaryDetails()` method in the interface.

3. Create a `RegularEmployee` class that implements the `Employee` interface.
4. Implement `salaryDetails()` to calculate and display the regular employee's salary.
5. Create a `ContractEmployee` class that implements the `Employee` interface.
6. Implement `salaryDetails()` to calculate and display the contract employee's salary.
7. Create a `Vendor` class that implements the `Employee` interface.
8. Implement `salaryDetails()` to display the vendor salary message.
9. Read the employee ID from the user.
10. Create an `Employee` interface reference.
11. Check the first character of the employee ID using `startsWith()`.
12. If the ID starts with `R`, create a `RegularEmployee` object.
13. If the ID starts with `C`, create a `ContractEmployee` object.
14. If the ID starts with `V`, create a `Vendor` object.
15. If the ID is invalid, display an error message.
16. Call `salaryDetails()` using the `Employee` reference.
17. The appropriate implementation is executed at runtime.



UML Class Diagram



Relationship: `RegularEmployee`, `ContractEmployee`, and `Vendor` implement the `Employee` interface.



Source Code

```
import java.util.Scanner;

// creating an Employee interface
interface Employee {

    void salaryDetails();
}

// creating a class for regular employee
class RegularEmployee implements Employee {

    // implementing salaryDetails method
    public void salaryDetails() {

        int basic = 25000;
        int hra = 15000;
        int ta = 5000;
        int total = basic + hra + ta;

        System.out.println("Salary Details:");
        System.out.println("Basic Pay: " + basic);
        System.out.println("HRA: " + hra);
        System.out.println("T.A: " + ta);
        System.out.println("Total Amount: " + total);
    }
}

// creating a class for contract employee
class ContractEmployee implements Employee {

    // implementing salaryDetails method
    public void salaryDetails() {

        int basic = 12000;
        int hra = 0;
        int ta = 3000;
        int total = basic + hra + ta;

        System.out.println("Salary Details:");
        System.out.println("Basic Pay: " + basic);
```

```

        System.out.println("HRA: " + hra);
        System.out.println("T.A: " + ta);
        System.out.println("Total Amount: " + total);
    }
}

```

```

// creating a class for vendor
class Vendor implements Employee {

    // implementing salaryDetails method
    public void salaryDetails() {

        System.out.println(
            "Vendor salary details are not available."
        );
    }
}

```

```

// main class
public class EmployeeSalary {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // taking employee id from user
        System.out.print("Enter Employee Id: ");
        String id = sc.nextLine();

        // creating interface reference
        Employee employee;

        // checking employee id
        if (id.startsWith("R")) {

            // creating regular employee object
            employee = new RegularEmployee();
        }

        else if (id.startsWith("C")) {

            // creating contract employee object

```

```
        employee = new ContractEmployee();
    }

    else if (id.startsWith("V")) {

        // creating vendor object
        employee = new Vendor();
    }

    else {

        System.out.println("Invalid Employee Id");
        sc.close();
        return;
    }

    // calling method using interface reference
    employee.salaryDetails();

    sc.close();
}
}
```

Compilation & Execution Commands

Save the source code exactly as:

EmployeeSalary.java

Compile the program:

```
javac EmployeeSalary.java
```

Run the program:

```
java EmployeeSalary
```



Output Screen

Example 1: Regular Employee

Enter Employee Id: R101

Salary Details:

Basic Pay: 25000

HRA: 15000

T.A: 5000

Total Amount: 45000

Example 2: Contract Employee

Enter Employee Id: C101

Salary Details:

Basic Pay: 12000

HRA: 0

T.A: 3000

Total Amount: 15000

Example 3: Vendor

Enter Employee Id: V101

Vendor salary details are not available.

Example 4: Invalid Employee ID

Enter Employee Id: X101

Invalid Employee Id

```
PS C:\Users\ navee\OneDrive\Desktop\programs>
javac Week8.java
PS C:\Users\ navee\OneDrive\Desktop\programs> java Week8
Enter Employee Id: R
Salary Details:
Basic Pay: 25000
HRA: 15000
T.A: 5000
Total Amount: 45000
```